Arquitectura de un sistema de chat en AWS

1. Requerimientos

1.1 Funcionales

#	Requerimiento
F1	Registro e inicio de sesión con correo + contraseña (verificación de email)
F2	Federación con IdPs externos: Google, Apple, Facebook, Microsoft (OIDC/SAML)
F3	Perfil de usuario (nombre visible, avatar, estado)
F4	Búsqueda/invitación de contactos por email o username
F5	Chat 1 a 1
F6	Chat de grupo, máximo 100 miembros, con roles (owner/admin/member)
F7	Envío/recepción de mensajes de texto en tiempo real
F8	Historial paginado y persistente por conversación
F9	Estados de mensaje: enviado → entregado → leído
F10	Contador de no leídos por conversación y lista de conversaciones ordenada por actividad
F11	Presencia (online/offline/última conexión) e indicador "escribiendo…"
F12	Sincronización multi-dispositivo y catch-up tras reconexión
F13	Notificaciones fuera de sesión (Web Push / email digest)
F14	Moderación básica: bloquear usuario, reportar, salir/eliminar grupo
F15	Borrado de cuenta y exportación de datos (cumplimiento)

1.2 No funcionales

Atributo	Objetivo
Disponibilidad	99.95 % mensual (≈22 min/mes) en la capa de mensajería
Latencia	p99 < 300 ms extremo a extremo (emisor → receptor conectado, misma región); p99 < 150 ms en API de historial
Durabilidad	0 pérdida de mensajes confirmados (ACK solo después de persistir)
Consistencia	Orden total por conversación (secuencia monotónica); eventual entre conversaciones
Escalabilidad	Horizontal y elástica; sin cambios de arquitectura hasta 10× el tráfico inicial
Idempotencia	Reintentos del cliente no duplican mensajes (clientMessageId)

Atributo	Objetivo
Seguridad	TLS 1.2+ en tránsito, KMS en reposo, JWT de Cognito en cada request, WAF en el borde, mínimo privilegio en IAM
Privacidad	Cifrado en reposo por defecto; retención configurable; derecho al olvido
Observabilidad	Métricas, logs estructurados y trazas distribuidas; alarmas sobre latencia de entrega y DLQ
Costo	Modelo pay-per-use, costo marginal decreciente por usuario
RTO / RPO	RTO 4 h, RPO 1 min (DR activo-pasivo multi-región)
Compatibilidad	Navegadores modernos (Chrome, Edge, Firefox, Safari); WCAG 2.1 AA en la UI

2. Estimaciones de capacidad (back-of-the-envelope)

Supuestos base (año 1, objetivo de diseño):

```
Usuarios registrados (MAU)        1,000,000
DAU (20 % de MAU)                   200,000
Sesiones/día por DAU                      3   (≈ 25 min c/u)
Mensajes enviados por DAU/día            40
Tamaño medio de mensaje             100 B de texto
Ratio conversaciones                80 % 1:1  /  20 % grupo (tamaño medio 20)
```

Tráfico de escritura

```
Mensajes/día  = 200,000 × 40            = 8,000,000
QPS promedio  = 8M / 86,400             ≈ 93 msg/s
Factor pico (hora punta ≈ 4×)           ≈ 370 msg/s  → diseñar para 500 msg/s
```

Fan-out (entregas)

Destinatarios promedio = 0.8×1 + 0.2×20 ≈ 4.8 Entregas/día = 8M × 4.8 ≈ 38,400,000 Entregas pico = 500 × 4.8 ≈ 2,400 entregas/s

Conexiones concurrentes

Concurrencia = DAU × tiempo conectado / ventana activa ≈ 200,000 × 75 min/día / 8 h ≈ 30,000 conexiones WebSocket concurrentes en pico

Ancho de banda

Payload por entrega (JSON + framing) ≈ 1 KB Pico = 2,400 × 1 KB ≈ 2.4 MB/s ≈ 20 Mbps → despreciable

Almacenamiento

Ítem de mensaje en DynamoDB ≈ 400 B (PK/SK + metadatos + texto) Por día = 8M × 400 B ≈ 3.2 GB/día Por año ≈ 1.17 TB 5 años (hot 90 días en DynamoDB, resto en S3): DynamoDB hot ≈ 290 GB S3 archivo (Parquet + gzip, ~4:1) ≈ 1.4 TB Metadatos (usuarios, membresías, conversaciones) = 1M × 2 KB + 5M × 300 B ≈ 3.5 GB

Capacidad DynamoDB

```
Escrituras por mensaje = 1 (ítem MSG, 1 KB → 1 WRU)
                       + 1 (UpdateItem ADD lastSeq/lastMessageAt en CONV#META)
                       = 2 WRU
```

```
Pico                    = 500 msg/s × 2 = 1,000 WRU/s

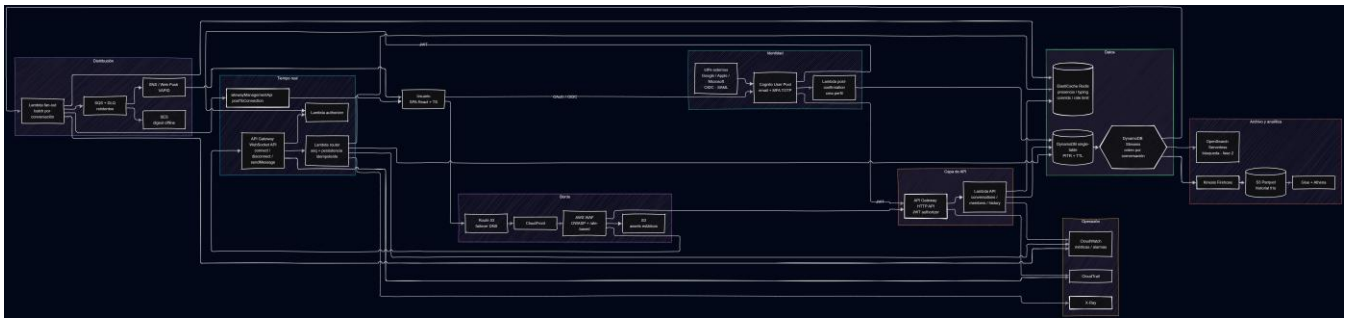
Lecturas: apertura de conversación = Query de 50 ítems ≈ 7 RCU (eventual)
  200,000 DAU × 3 sesiones × 3 conversaciones = 1.8M queries/día
  ≈ 21 q/s promedio, ~85 q/s pico ≈ 600 RCU/s
```

Muy por debajo de los límites de partición (1,000 WCU / 3,000 RCU). El contador lastSeq de una conversación de 100 miembros nunca se acerca a 1,000 escrituras/s sobre el mismo ítem.

Orden de magnitud de costo: con on-demand, los rubros dominantes son mensajes y minutos de conexión de API Gateway WebSocket (~1.2 B mensajes/mes), DynamoDB (~500 M WRU/mes) y Lambda; el rango esperado es de miles de USD al mes, no decenas de miles. Para cifras confiables usa la AWS Pricing Calculator (https://calculator.aws) con estos volúmenes. A partir de ~6 meses de tráfico estable conviene mover DynamoDB a capacidad provisionada con auto scaling y comprar Compute Savings Plans para Lambda/Fargate.

3. Arquitectura de alto nivel

Proveedor: AWS, región primaria us-east-1 (o la más cercana a los usuarios), con DR en us-west-2. Enfoque serverless-first: la carga de chat es muy variable (picos de 4×) y el modelo pay-per-use evita sobreaprovisionar 30k conexiones concurrentes.



3.1 Capa de borde y frontend

- SPA (React/TypeScript) empaquetada y servida desde S3 + CloudFront (OAC, sin bucket público), HTTP/2-3, Cache-Control inmutable para assets con hash.
- AWS WAF en CloudFront: managed rules (OWASP), rate-based rule por IP, bot control.
- Route 53 con health checks y failover DNS hacia la región DR.

3.2 Autenticación e identidad

- Cognito User Pool: registro por email con verificación por código, política de contraseñas, MFA opcional (TOTP), detección de credenciales comprometidas.
- Federación: Google/Apple/Microsoft vía OIDC, y cualquier SAML corporativo. El User Pool unifica identidades → un solo sub por usuario.
- Tokens: accessToken (1 h) + refreshToken (30 d). El JWT se valida en API Gateway (JWT authorizer) y en $connect del WebSocket (Lambda authorizer). El WebSocket recibe el token como query param firmado o vía cookie HttpOnly+Secure; se re-valida la expiración en cada mensaje para forzar reconexión.
- Lambda trigger post-confirmation → crea el perfil en DynamoDB de forma idempotente.

3.3 API de aplicación (request/response)

API Gateway HTTP API + Lambda para todo lo que no es tiempo real:

```
POST   /v1/conversations                 crear 1:1 o grupo
GET    /v1/conversations?cursor=         inbox ordenado por lastMessageAt
POST   /v1/conversations/{id}/members    agregar (condicional: memberCount < 100)
DELETE /v1/conversations/{id}/members/me salir
GET    /v1/conversations/{id}/messages?afterSeq=|beforeSeq=&limit=50
POST   /v1/conversations/{id}/read       { lastReadSeq }
```

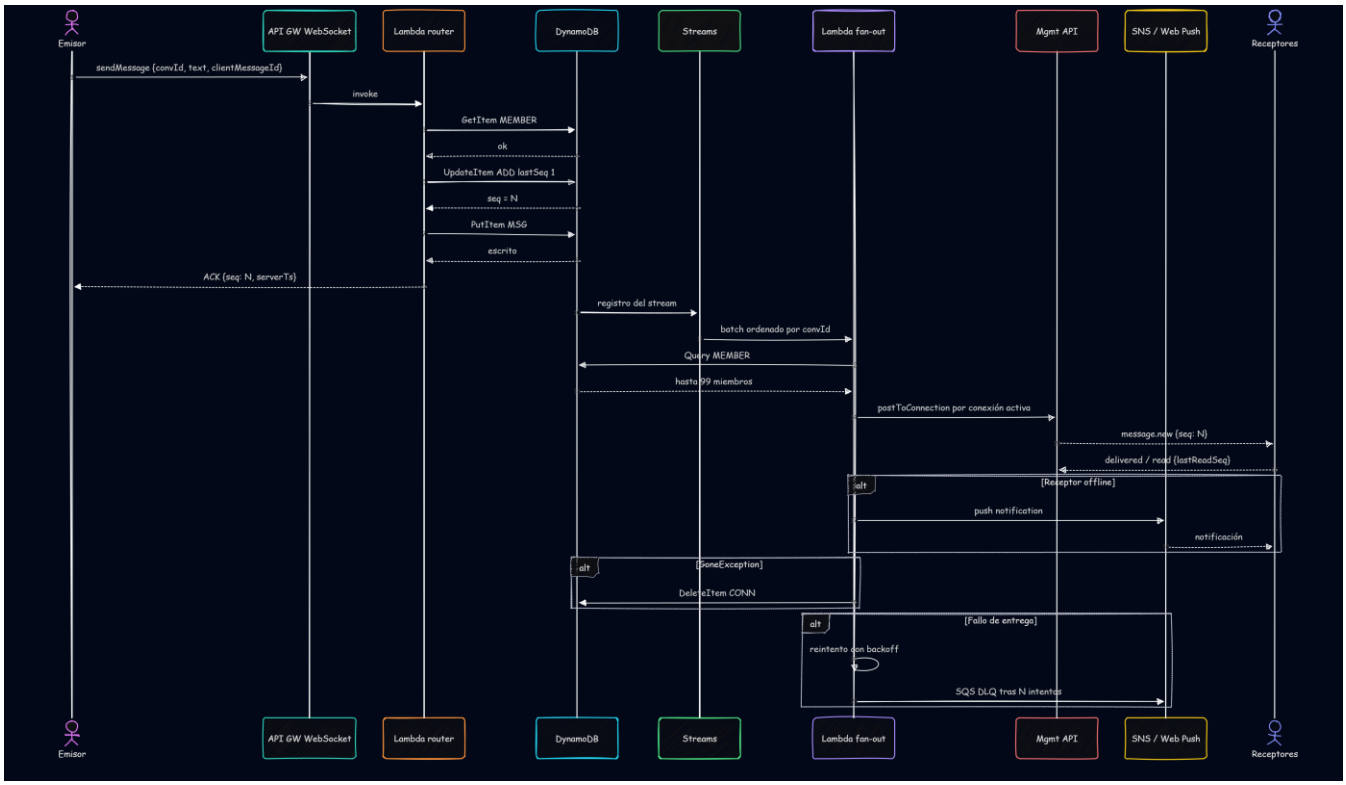
```
GET    /v1/users/me | PATCH /v1/users/me
```

El límite de 100 miembros se aplica con UpdateItem ... ADD memberCount 1 + ConditionExpression: memberCount < 100: atómico y sin condiciones de carrera.

3.4 Capa de tiempo real

API Gateway WebSocket API + Lambda es la opción recomendada: conexiones gestionadas, sin servidores que mantener, facturación por mensaje y por minuto de conexión.

- $connect: autoriza el JWT, escribe CONN#<connectionId> en DynamoDB (con TTL de seguridad) y registra connId en el set de Redis del usuario.
- $disconnect: limpia registros y marca lastSeenAt.
- Ruta sendMessage: valida membresía, obtiene el seq siguiente con UpdateItem ADD lastSeq 1 sobre CONV#META, persiste el mensaje condicionado por clientMessageId (idempotencia) y devuelve ACK al emisor con {seq, serverTs}.
- DynamoDB Streams → Lambda fan-out: resuelve los ≤99 destinatarios, agrupa por conexión y llama postToConnection. GoneException → borra la conexión huérfana. Fallos → SQS con DLQ y reintentos.
- Destinatarios offline → SNS/Web Push (VAPID) y, opcionalmente, digest por SES.
- Presencia y typing viven solo en Redis con TTL de 30–60 s: son eventos efímeros de alto volumen que no deben tocar la base de datos ni el stream.
- Catch-up: el cliente guarda lastSeq por conversación; al reconectar hace GET /messages?afterSeq=N. Los huecos en la secuencia son detectables, lo que hace la sincronización determinista.

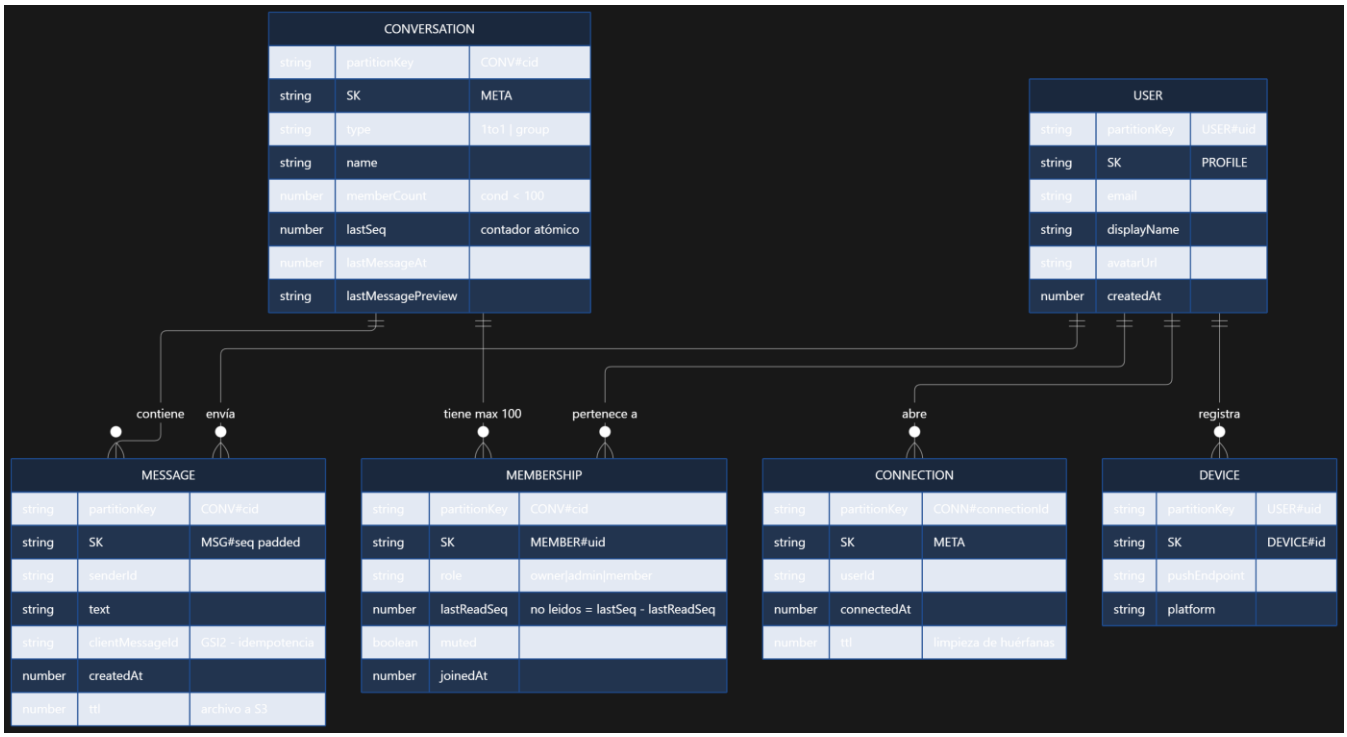


Alternativas evaluadas: AWS AppSync (suscripciones GraphQL) reduce el código de fan-out y es excelente si el equipo ya usa GraphQL, pero da menos control sobre enrutamiento y payload. ECS Fargate/EKS con WebSocket propio + Redis Pub/Sub conviene si se necesitan >200k conexiones concurrentes, protocolo binario propio o E2EE con estado en servidor; el costo es operar el fleet, el sticky routing y el autoscaling por conexiones.

3.5 Modelo de datos (DynamoDB single-table)

Entidad	PK	SK	Atributos

Entidad	PK	SK	Atributos
Usuario	USER#<uid>	PROFILE	email, displayName, avatarUrl, createdAt
Conversación	CONV#<cid>	META	type(1:1|group), name, memberCount, lastSeq, lastMessageAt, lastMessagePreview
Miembro	CONV#<cid>	MEMBER#<uid>	role, joinedAt, lastReadSeq, muted
Inbox del usuario	USER#<uid>	CONV#<cid>	lastMessageAt (GSI1 SK), pinned
Mensaje	CONV#<cid>	MSG#<seq padded>	senderId, text, createdAt, clientMessageId, TTL
Conexión	CONN#<connId>	META	userId, connectedAt, ttl
Dispositivo	USER#<uid>	DEVICE#<id>	pushEndpoint, platform



- GSI1 (userId → lastMessageAt): inbox ordenado en una sola Query. Es una proyección de MEMBERSHIP, no una entidad independiente.
- GSI2 (clientMessageId): deduplicación y auditoría.
- Mensajes ordenados por seq con padding: Query descendente paginada nativa, sin escaneos.
- TTL en mensajes (p. ej. 365 d) + Streams → Firehose → S3 Parquet como archivo consultable con Athena. La tabla operativa se mantiene pequeña y barata sin perder historia.

3.6 Caché

- ElastiCache for Redis (cluster mode, Multi-AZ): presencia, sets user → connIds, typing, rate limiting por usuario (token bucket), caché de metadatos de conversación y de la lista de miembros de grupos activos (evita una Query de 100 ítems por cada mensaje).
- DAX es opcional y solo si el patrón de lectura de historial se vuelve dominante; en el diseño actual Redis cubre el caso caliente.

3.7 Escalabilidad

- Lambda escala por concurrencia: usar concurrencia reservada en sendMessage y fan-out para proteger la base de datos, y provisioned concurrency en rutas críticas para eliminar cold starts en horas punta.
- DynamoDB: on-demand al inicio → provisionado con auto scaling cuando el tráfico sea predecible. La clave de partición (CONV#<cid>) distribuye naturalmente.
- Cuellos de botella conocidos y mitigación: cuotas de API Gateway WebSocket (nuevas conexiones/s y conexiones concurrentes son límites ajustables — solicitar aumento antes del lanzamiento), tamaño de batch del stream (ajustar BatchSize/MaximumBatchingWindow, usar ParallelizationFactor) y el ítem lastSeq de conversaciones muy activas (sharding del contador si alguna vez supera ~1,000 escrituras/s).
- Grupos de 100 son un fan-out trivial; el diseño soporta grupos mucho mayores sin cambios estructurales si el requisito crece.

3.8 Disponibilidad y DR

- Todo en ≥3 AZ (los servicios gestionados ya son Multi-AZ; Redis con réplica en otra AZ y failover automático).
- DynamoDB Global Tables hacia la región DR → RPO de segundos. Cognito con export periódico de usuarios (limitación conocida: las contraseñas no se replican; los usuarios federados re-autentican sin fricción).
- Route 53 failover + pipeline de IaC capaz de desplegar el stack completo en la región secundaria (activo-pasivo, RTO 4 h). Backups PITR de DynamoDB (35 días) y versionado + Object Lock en el bucket de archivo.

3.9 Seguridad

- IAM por función con permisos mínimos; ninguna Lambda puede escribir fuera de sus claves.
- KMS CMK para DynamoDB, S3, SQS y Redis (cifrado en reposo y en tránsito).
- Autorización de dominio: toda operación sobre una conversación verifica la existencia del ítem MEMBER#<uid>; nunca se confía en el conversationId enviado por el cliente.
- Sanitización del texto en el servidor; el cliente renderiza como texto plano (no innerHTML) para evitar XSS almacenado.
- Rate limiting en dos niveles: WAF/API Gateway (por IP) y Redis (por usuario).
- Secretos en Secrets Manager / SSM Parameter Store; nada en variables de entorno en claro.
- Auditoría con CloudTrail; GuardDuty y Security Hub activos.

3.10 Observabilidad y entrega

- CloudWatch con métricas custom: message_publish_to_deliver_ms (p50/p99), entregas fallidas, profundidad de DLQ, conexiones activas, tasa de reconexión.
- X-Ray para trazas extremo a extremo; logs JSON estructurados con correlationId.
- Alarmas → SNS → on-call; dashboards por dominio.
- CDK (TypeScript) como IaC; CI/CD en CodePipeline o GitHub Actions con despliegues canary (Lambda aliases + CodeDeploy) y entornos dev/stage/prod aislados por cuenta (AWS Organizations).

4. Decisiones de diseño y sus contrapartidas

Tres decisiones cargan el peso de esta arquitectura. Cada una resuelve un problema concreto y cobra un precio que conviene tener explícito.

4.1 DynamoDB Streams como columna vertebral del fan-out

Qué resuelve. El mensaje se confirma al emisor después de persistir, y la distribución se dispara desde el registro ya escrito. Eso convierte "mensaje persistido" en la única fuente de verdad: no existe un estado en el que el receptor haya visto algo que la base de datos no tenga. Además, el stream ordena los registros por clave de partición —es decir, por conversación— y aporta reintentos, ParallelizationFactor y DLQ sin código propio.

Qué cuesta. Un salto extra en la ruta crítica: el polling del stream añade decenas a un par de cientos de milisegundos al p99 de entrega. La alternativa (que el Lambda router haga postToConnection directo) es más rápida pero obliga a implementar a mano el orden, los reintentos y la recuperación ante fallos parciales.

Si el presupuesto de latencia aprieta. Esquema híbrido: envío optimista directo desde el router para los destinatarios conectados, y el stream como camino de garantía y reconciliación. El cliente deduplica por seq, así que recibir el mismo mensaje por ambas vías es inocuo.

4.2 Secuencia monotónica por conversación (seq)

Qué resuelve. Un entero incremental por conversación convierte la sincronización multi-dispositivo en algo determinista en vez de heurístico: el cliente pide afterSeq=N y sabe exactamente qué le falta; los huecos en la serie son detectables; la paginación del historial es un cursor natural; y el acuse de lectura es un solo entero (lastReadSeq) en lugar de una lista de IDs.

Qué cuesta. Una escritura adicional por mensaje (el UpdateItem ADD sobre CONV#META) y un techo de ~1,000 escrituras/s sobre ese único ítem. Para grupos de 100 miembros está a órdenes de magnitud de distancia; si alguna vez hiciera falta, se resuelve con un contador fragmentado.

Alternativa descartada. IDs basados en tiempo (ULID/KSUID) evitan la escritura extra, pero dependen del reloj, admiten empates y no permiten detectar huecos: la sincronización pasa a ser "tráeme lo posterior a este timestamp y espero que no se haya perdido nada".

4.3 No materializar contadores de no leídos

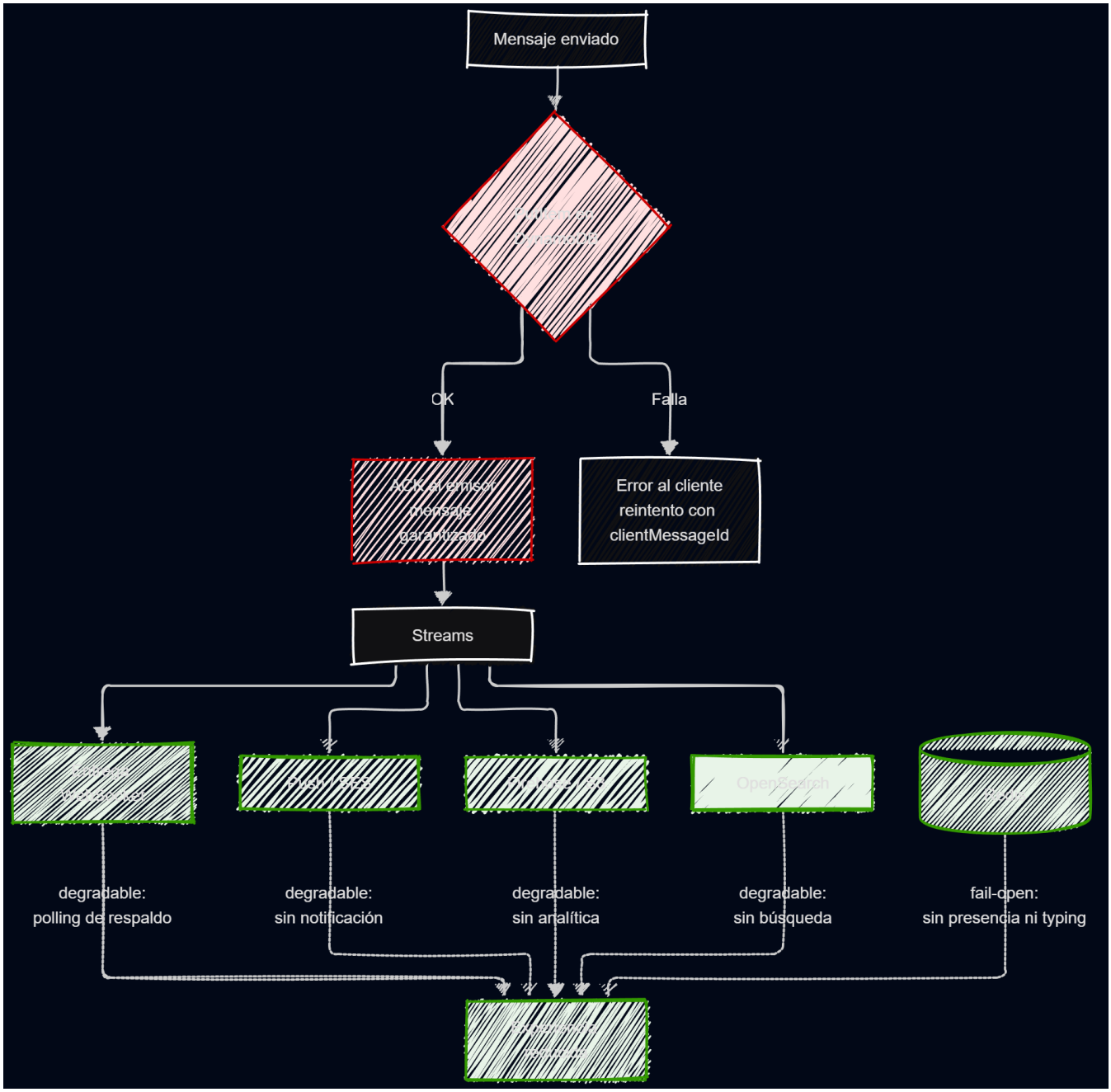
Qué resuelve. Mantener un contador por miembro implicaría, en un grupo de 100, cien escrituras por mensaje: a 500 mensajes/s de pico serían hasta ~50,000 WRU/s solo para los contadores, frente a los 1,000 WRU/s del diseño actual. Almacenando lastReadSeq en el ítem de membresía y calculando no_leídos = conv.lastSeq − member.lastReadSeq en tiempo de lectura, el fan-out de escrituras O(N) desaparece por completo.

Qué cuesta. Renderizar la bandeja de entrada necesita dos fuentes: una Query sobre GSI1 para las membresías del usuario y los lastSeq de esas conversaciones. Se resuelve con un BatchGetItem de los ~20–30 CONV#META visibles (1 RCU cada uno), cacheado en Redis e invalidado por el propio fan-out. Es una lectura barata y acotada a cambio de eliminar el punto de escalado más peligroso del sistema.

5. Resiliencia y tolerancia a fallos

5.1 Principio rector: un solo camino crítico

La única dependencia dura para que un mensaje no se pierda es DynamoDB. Todo lo demás —Redis, push, analítica, búsqueda— es degradable. Esa jerarquía se hace explícita en el código: el ACK al emisor se emite después del PutItem y de nada más; si Redis, SNS o Firehose fallan, el mensaje ya está a salvo y el sistema sigue operando en modo reducido.



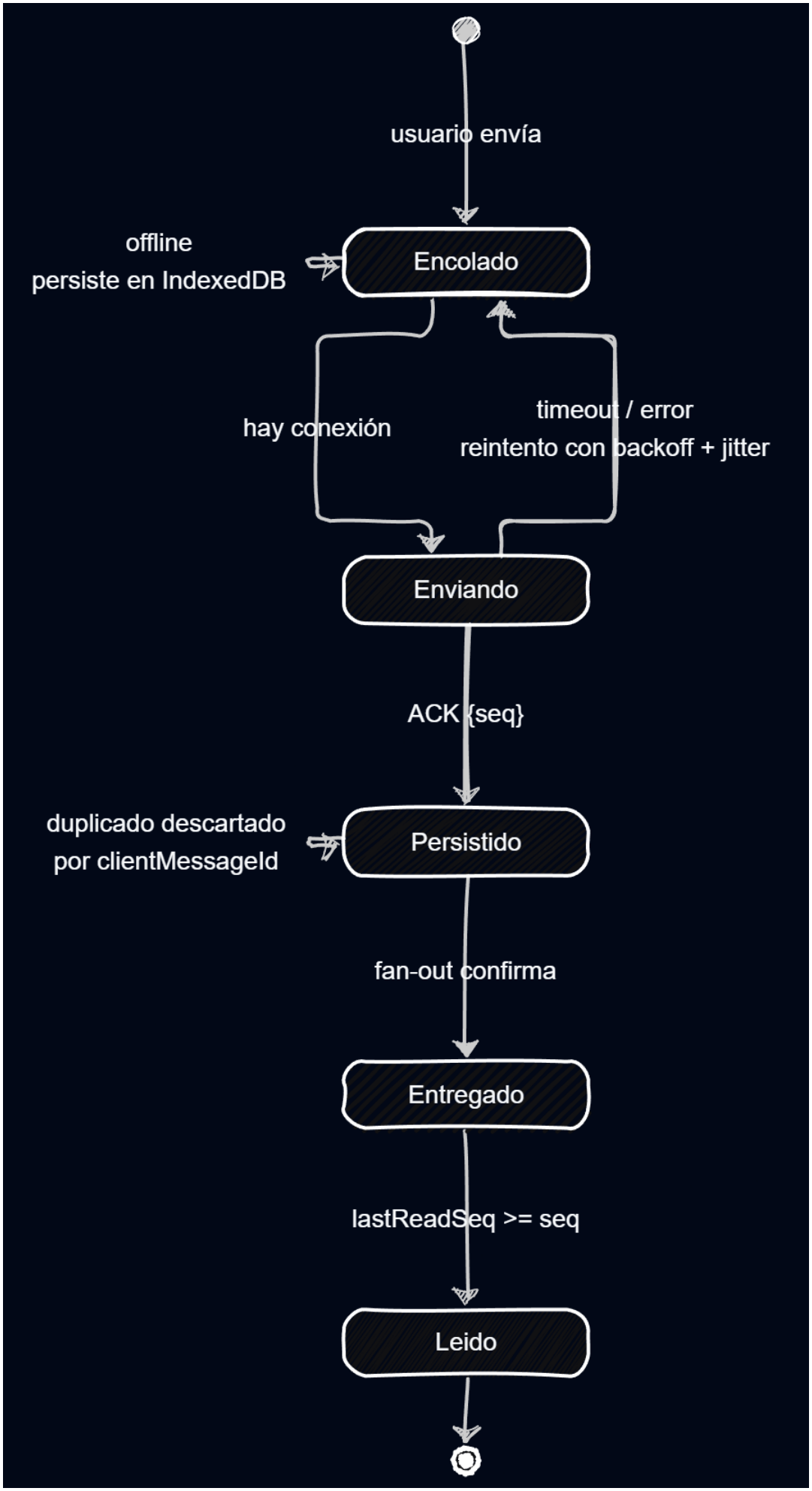
5.2 Modos de fallo y mitigación

Fallo	Detección	Mitigación	Impacto residual
Pérdida de una AZ	CloudWatch + health checks	Servicios gestionados son Multi-AZ; Redis con réplica y failover automático	Reconexión masiva de WebSockets (ver 5.5)
Pérdida de región	Alarmas compuestas + canarios	Route 53 failover a DR; Global Tables ya replicando	RTO 4 h; sesiones Cognito requieren re-login
Stream estancado (Lambda fan-out falla)	Alarma sobre IteratorAge > 60 s	ReportBatchItemFailures + BisectBatchOnFunctionError; OnFailure → SQS	Entrega retrasada, no perdida; el catch-up del cliente lo resuelve
Poison pill (registro que siempre falla)	IteratorAge creciente + DLQ con contenido	Fallo parcial por ítem: solo ese registro se reintenta, el shard no se bloquea	Un mensaje retrasado, no una conversación congelada

Fallo	Detección	Mitigación	Impacto residual
Throttling de DynamoDB	ThrottledRequests, SystemErrors	On-demand absorbe picos; reintentos con backoff del SDK; concurrencia reservada como amortiguador	Latencia elevada puntual
Throttling de Lambda	Throttles, ConcurrentExecutions	Concurrencia reservada por función para que el fan-out no ahogue a la API ni viceversa	Degradación acotada a una función
Caída de Redis	CacheHitRate, errores de conexión	Fail-open: presencia y typing se desactivan; el fan-out lee miembros desde DynamoDB	Se pierden funciones cosméticas; mensajería intacta
Cuota de API Gateway WebSocket	ClientError 429, conexiones rechazadas	Cuotas elevadas por adelantado; reconexión con jitter	Reconexiones más lentas en eventos masivos
Caída de Cognito	Canario de login	Tokens vigentes (1 h) siguen funcionando; solo se bloquean logins nuevos	Usuarios ya conectados no se enteran
Conexiones huérfanas (GoneException)	Error en postToConnection	Borrado inmediato del ítem + TTL como red de seguridad	Ninguno
Entrega duplicada	—	At-least-once por diseño; el cliente deduplica por seq	Ninguno

5.3 Idempotencia y garantías de entrega

- Escritura: clientMessageId generado por el cliente + ConditionExpression. Un reintento por timeout de red no crea un segundo mensaje.
- Entrega: at-least-once. El cliente mantiene el seq máximo por conversación y descarta lo repetido; nunca asume exactly-once desde la red.
- Lectura: afterSeq es la única primitiva de sincronización. Tras cualquier fallo —reconexión, cambio de dispositivo, stream retrasado— el cliente converge pidiendo lo que le falta, sin depender de que la entrega en vivo haya funcionado.
- Cola de salida en el cliente: los mensajes pendientes se persisten en IndexedDB y se reenvían al reconectar. El usuario puede escribir sin conexión y el estado "enviando" es real, no cosmético.


usuario envía
offline
persiste en IndexedDB
Encolado
hay conexión
timeout / error
reintento con backoff + jitter
Enviando
ACK {seq}
duplicado descartado
por clientMessageId
Persistido
fan-out confirma
Entregado
lastReadSeq >= seq
Leido

5.4 Degradación gradual

Situación	Comportamiento
WebSocket no disponible	Fallback a polling GET /messages?afterSeq= cada 5–10 s
Redis caído	Sin presencia ni typing; banner discreto de "funciones limitadas"
Fan-out retrasado	Entrega diferida; el badge de no leídos se actualiza al abrir la app
Push/SES caídos	Notificaciones fuera de sesión se pierden; el historial no
OpenSearch caído	Búsqueda deshabilitada; chat sin cambios

5.5 Reconexión masiva (thundering herd)

Es el riesgo operativo más subestimado de una arquitectura WebSocket. Si se cae una AZ o se despliega mal, 30,000 clientes intentan reconectar en el mismo segundo contra un límite de nuevas conexiones por segundo. Mitigación en tres capas:

1. Cliente: backoff exponencial con jitter completo (aleatorio entre 0 y el intervalo), no backoff fijo. Sin jitter, los clientes se resincronizan y golpean en oleadas.
2. Servidor: el $connect devuelve Retry-After con un desfase calculado cuando detecta saturación, distribuyendo el reingreso en una ventana de minutos.
3. Cuotas: aumento solicitado con antelación y dimensionado para reconectar la base completa en < 3 min.

5.6 Despliegues y cambios

- Canary con Lambda aliases + CodeDeploy (10 % → 100 % con rollback automático por alarma de errores o latencia).
- Cambios de esquema hacia adelante: nuevos atributos opcionales, nunca renombrados en caliente. El single-table design tolera versiones convivientes.
- Compatibilidad de protocolo: el payload de WebSocket lleva v; el servidor soporta N y N-1 mientras se propaga la SPA (los navegadores con pestañas viejas abiertas son clientes desactualizados persistentes).
- Feature flags (AppConfig) para apagar funciones caras o inestables sin desplegar.

5.7 Verificación de la resiliencia

No se declara tolerante a fallos lo que no se ha roto a propósito:

- Canarios sintéticos (CloudWatch Synthetics) cada minuto: login real → abrir WebSocket → enviar mensaje → verificar recepción en un segundo cliente. Mide el SLO extremo a extremo, no la salud de componentes aislados.
- AWS FIS (game days trimestrales): inyectar errores y latencia en Lambda, forzar failover de Redis, provocar throttling de DynamoDB, cortar una AZ.
- Simulacro de restauración: restaurar PITR a una tabla nueva y validar integridad; un backup no probado no es un backup.
- Ensayo de DR: failover completo a la región secundaria con medición real de RTO.
- Presupuesto de error: 99.95 % permite ~22 min/mes. Si se consume la mitad, se congelan features y la siguiente iteración va a fiabilidad.

5.8 Alarmas que importan

Cuatro señales detectan casi cualquier degradación real, frente a decenas de métricas de infraestructura que no correlacionan con la experiencia:

1. IteratorAge del stream — salud del fan-out.

2. message_publish_to_deliver_ms p99 — el SLO del usuario.

3. Profundidad y edad de la DLQ — mensajes en riesgo.

4. Tasa de fallo del canario sintético — ¿funciona el flujo completo?

6. Optimización de costos

6.1 Rubros dominantes

En orden de peso esperado a 8 M mensajes/día:

1. API Gateway WebSocket — mensajes entregados (~1.2 B/mes) + minutos de conexión (~1.3 B/mes). El fan-out multiplica: cada mensaje enviado se factura ~4.8 veces como entrega.

2. DynamoDB — ~500 M WRU/mes más almacenamiento.

3. Lambda — invocaciones de router y fan-out.

4. CloudWatch Logs — habitualmente el costo sorpresa (ver 6.3).

5. Cognito — revisar con cuidado: los niveles avanzados de seguridad se facturan por MAU, y a 1 M de MAU ese rubro puede superar al resto del sistema junto. Evaluar el nivel mínimo necesario y activar protección avanzada de forma selectiva.

6. ElastiCache — costo fijo por hora, independiente del tráfico.

6.2 Palancas de arquitectura (las que más rinden)

Reducir entregas facturadas, no solo su precio

- Agrupar frames en ráfagas: durante picos, varios mensajes de la misma conversación viajan en un solo postToConnection (ventana de 100–200 ms). En grupos activos recorta entregas facturadas de forma directa sin afectar el p99 percibido.
- Payload mínimo: enviar el delta (convId, seq, senderId, text), no el objeto completo con metadatos que el cliente ya tiene. Mantiene cada mensaje en 1 unidad de facturación y reduce el ancho de banda.
- No difundir lo efímero: typing y presencia se agregan en Redis y se emiten con throttling (máximo 1 evento cada 3 s por usuario). Sin esta restricción, el tráfico de "escribiendo…" puede superar al de mensajes reales.

Reducir minutos de conexión

- Cerrar el WebSocket cuando la pestaña lleva >5 min oculta y confiar en Web Push para despertar. En un cliente web, las pestañas abandonadas son la mayor fuente de minutos de conexión inútiles.

Reducir WRU en DynamoDB

- No materializar contadores de no leídos (decisión 4.3): evita ~50× escrituras en grupos. Es, con diferencia, la decisión de mayor impacto económico del diseño.
- Mantener el ítem de mensaje bajo 1 KB: nombres de atributo cortos (t, s, c), sin campos redundantes. Cruzar el límite de 1 KB duplica el costo de escritura de todo el sistema.
- Proyección KEYS_ONLY o INCLUDE en los GSI: una proyección ALL duplica el costo de cada escritura.
- Streams con NEW_IMAGE solo si el fan-out realmente lo necesita.

6.3 Palancas operativas

Palanca	Acción	Efecto típico

Palanca	Acción	Efecto típico
Logs	Nivel WARN en producción, muestreo al 1–5 % en rutas calientes, retención 7–14 días en CloudWatch y archivo a S3	La ingesta de logs de un sistema de 8 M msg/día puede alcanzar cientos de GB/mes; reducción de dos dígitos porcentuales sobre la factura total
Graviton	arm64 en todas las Lambdas	~20 % menos por GB-s
Memoria de Lambda	Ajustar con Lambda Power Tuning	Más memoria a veces cuesta menos por ejecutar más rápido
NAT Gateway	Evitarlo: Gateway Endpoints (gratis) para DynamoDB y S3; Interface Endpoints solo donde haga falta	Un NAT innecesario para Lambdas en VPC cuesta cientos de USD/mes en horas + procesamiento de datos
ElastiCache	Dimensionar por memoria real; considerar Redis Serverless mientras el tráfico sea irregular	Elimina pago por capacidad ociosa nocturna
Firehose	Buffer amplio (5 min / 64–128 MB) y Parquet + compresión	Menos PUTs a S3 y ~4:1 en almacenamiento; además abarata Athena, que cobra por bytes escaneados
S3	Lifecycle: Standard → Intelligent-Tiering a 90 d → Glacier IR a 1 año; particionado por fecha	El historial frío tiende a costo casi nulo
TTL en DynamoDB	Expiración de mensajes (borrado gratuito) tras archivar en S3	Mantiene la tabla caliente en ~290 GB en vez de crecer 1.17 TB/año
Conexiones y dispositivos	TTL en CONN# y DEVICE#	Evita pagar almacenamiento y escaneos por basura acumulada

6.4 Modelo de compra

- Fase 1 (0–6 meses): todo on-demand. El tráfico es impredecible y el sobreaprovisionamiento cuesta más que la prima del on-demand.
- Fase 2 (tráfico estable): DynamoDB a capacidad provisionada con auto scaling (target 70 %) + capacidad reservada sobre la línea base; Compute Savings Plans para Lambda y Fargate; nodos reservados de ElastiCache.
- Revisión trimestral: el patrón de tráfico de un chat es muy estacional (día/noche, laboral/fin de semana), así que la base reservada se dimensiona sobre el valle, no sobre el promedio.

6.5 Gobierno del costo

- Etiquetado obligatorio (env, service, feature, team) aplicado por SCP o política de IaC; sin esto, el análisis posterior es imposible.
- AWS Budgets con alertas al 50/80/100 % y Cost Anomaly Detection sobre los rubros dominantes.
- Métrica de negocio, no de infraestructura: costo por DAU/mes y costo por millón de mensajes entregados, publicadas en el dashboard junto a los SLO. Con los supuestos de la sección 2, el objetivo razonable es del orden de centavos de USD por DAU al mes; cualquier desviación sostenida es un defecto de arquitectura, no un problema de facturación.
- Regla de decisión: antes de optimizar, medir qué porcentaje de la factura representa el rubro. Optimizaciones que ahorran <5 % del total casi nunca justifican la complejidad que añaden.
- Para cifras concretas de cualquiera de estos escenarios, modelarlos en la AWS Pricing Calculator (https://calculator.aws) con los volúmenes de la sección 2, y validar contra Cost Explorer una vez en producción.

7. Fuera de alcance (sin bloqueos arquitectónicos)

- Adjuntos: S3 + presigned URLs + escaneo antivirus.

- Búsqueda full-text: OpenSearch alimentado desde el mismo stream.
- E2EE: requiere fan-out por dispositivo (Double Ratchet) y renunciar a la búsqueda server-side y a la previsualización en notificaciones.
- Voz/video: Amazon Chime SDK o IVS.
- Apps móviles nativas: la API ya es agnóstica de cliente.